DATA ENGINEERING CASE STUDY

Engineering a Trusted Asset Management Data Pipeline

Candidate-facing practical assessment

Your challenge
Design and build a small working prototype that ingests asset-management data, validates it, produces trusted outputs, and demonstrates the engineering controls needed to operate the pipeline safely.

1. Scenario

An asset-management firm receives daily data for securities, transactions, positions, market prices and restricted securities. The data supports portfolio valuation, investment operations, risk, compliance and reporting. The goal is to replace fragile, manually controlled processing with a pipeline that is reliable, secure, observable and safe to rerun.

2. What you need to build

- Ingest Security Master, Transactions, Positions, Market Prices and Restricted Securities data.
- Validate technical and business data quality and explicitly handle rejected or quarantined records.
- Create a trusted portfolio-position output with market value and traceability metadata.
- Create a compliance-exception output for transactions executed while a security is under an active restriction.
- Make processing idempotent so reruns and duplicate deliveries do not double count business records.
- Produce run-level reconciliation and operational metrics.
- Demonstrate failure recovery, record-level lineage and a credible scaling approach.

3. Minimum input data

Dataset	Example fields
Security Master	security_id, isin, asset_class, currency, effective_from, effective_to
Transactions	transaction_id, portfolio_id, security_id, trade_date, quantity, price, currency
Positions	portfolio_id, security_id, position_date, quantity, book_value, currency
Market Prices	security_id, price_date, price, price_currency, price_source
Restricted Securities	security_id, restriction_type, effective_from, effective_to, status

Use synthetic data. You may extend the schemas where useful, but document your assumptions.

4. Engineering expectations

Area	What we expect you to demonstrate
Layering via medallion architecture	A clear Raw/Bronze -> Validated/Silver -> Curated/Gold approach or a justified equivalent.
Data quality	Missing keys, duplicates, unknown securities, invalid dates/prices, stale data and schema changes handled explicitly.
Temporal logic	Effective-dated Security Master and restricted-list joins where relevant.
Idempotency	Safe duplicate-file handling, corrected records and business-date reruns.
Reconciliation	Records received, accepted, rejected/quarantined, duplicates and key business exceptions.

Security	Least privilege, secrets protection, protected data access and auditable activity.
Observability	Pipeline health, freshness, volume, data-quality results, failures and useful alerts.
Recovery	No silent data loss or uncontrolled partial publication; provide retry/replay behaviour.
Scalability	Measure the prototype, identify the likely bottleneck and explain how you would scale it.

5. Required validation scenarios

- Duplicate delivery: process the same transaction input twice and show that positions are not double counted.
- Missing price: demonstrate the chosen reject/quarantine/warning behaviour and its effect on publication.
- Restricted security: create a trade during an active restriction period and generate a traceable exception.
- Mid-run failure: interrupt processing and demonstrate a safe restart or replay.
- Volume increase: run progressively larger synthetic datasets and discuss observed performance and scaling constraints.

6. Deliverables

- Architecture diagram showing sources, processing, storage, security, monitoring and consumption.
- Working source code plus synthetic test data or a generator.
- README with setup, execution, test and failure-injection instructions.
- Short engineering decision record covering keys, idempotency, late data, schema evolution, security, monitoring and scaling.
- A demonstration of a successful run, one controlled failure, safe replay, reconciliation and lineage.

7. Technology and constraints

Choose a technology stack that lets you demonstrate your engineering decisions clearly. Examples include Python, SQL, PySpark, Kafka, Airflow, dbt, DuckDB, PostgreSQL and Snowflake. A local implementation is acceptable.

- Use synthetic data only.
- AI-assisted development is permitted, but you must be able to explain and defend the implementation.
- Engineering judgement and principles matters.

What we will look for

Correctness, clear engineering decisions and principles, safe reruns, controlled failure handling, effective-dated business logic, operational visibility, security awareness, testability and the ability to explain trade-offs. Clearly state what you implemented and what you would add / change for production.